

Árvore B em Memória Secundária

Uma classe Árvore B de ordem m operando estritamente em disco

Algoritmos e Estruturas de Dados — AED-PG-2026 (5955001-3)

Prof. Dr. José Augusto Baranauskas — DCM / USP

C++17 · 1º Semestre/2026

Cássio Takarada · Cézio Luiz Ferreira Junior

Parte 1 — a estrutura

- Por que disco e por que Árvore B
- Anatomia do nó e do arquivo
- Busca, inserção e remoção
- Demonstração ao vivo

Parte 2 — a avaliação

- Experimentos e métricas
- Impacto da ordem m
- Escala e reaproveitamento
- Validação em 2 máquinas + conclusões

O problema

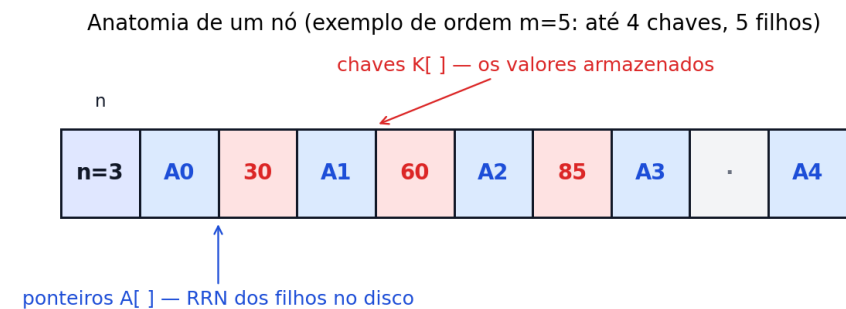
- Árvore B de ordem m que reside em disco (arquivo)
- Nunca carregada inteira — 1 nó por acesso
- Busca, inserção e remoção balanceadas
- Meta extra: avaliar m , escala e 2 máquinas

- RAM: rápida, pequena, volátil
- Disco: enorme, permanente, milhares de vezes mais lento
- Gargalo = disco, não CPU
- Objetivo: minimizar acessos ao disco

- Índice = sumário que evita ler o arquivo inteiro
- Árvore B: índice real de SGBDs e sistemas de arquivos
- Métrica honesta = nº de acessos ao disco (não o relógio)
- O relógio engana: a escrita vai ao cache do SO

Anatomia de um nó

- Registro de tamanho fixo, lido de uma vez
- Campos: n · ponteiros $A[]$ · chaves $K[]$
- Ordem $m \rightarrow$ até $m-1$ chaves e m filhos
- 1 nó = 1 acesso ao disco



Registro de tamanho fixo: 1 inteiro (n) + m ponteiros + m chaves = `PAGE_SIZE` bytes $\rightarrow$ 1 nó = 1 acesso ao disco

Layout do arquivo no disco

- Arquivo = vetor de registros (RRN)
- RRN 0 = header (root, total, free_head)
- $\text{offset} = \text{RRN} \times \text{PAGE_SIZE} \rightarrow$ acesso direto
- Nós removidos $\rightarrow$ lista de livres (reuso)

Layout do arquivo: um vetor de registros de tamanho fixo

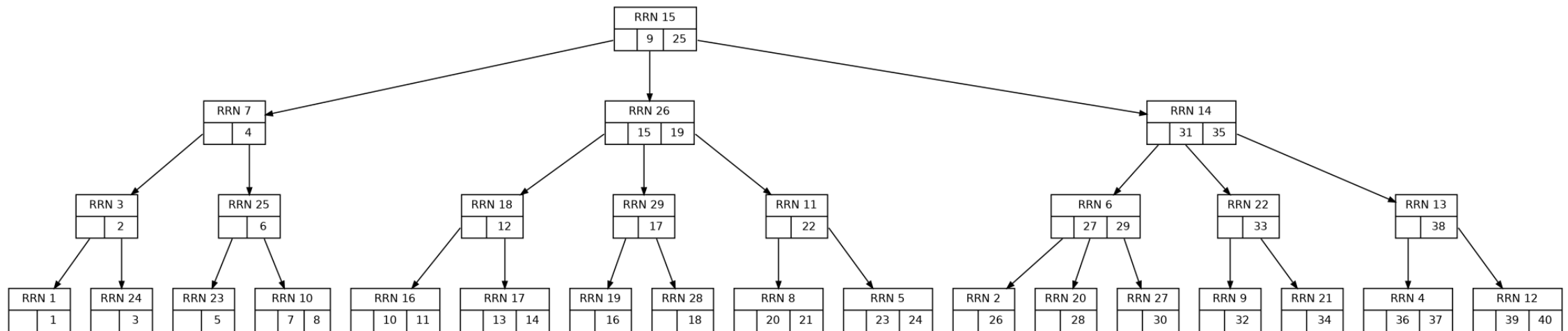


posição no arquivo: $\text{offset}(\text{rrn}) = \text{rrn} \times \text{PAGE_SIZE}$ (acesso direto, sem varrer nada)

free_head $\rightarrow$ lista de nós livres (reaproveitamento)

Decisões de implementação

- Ordem m = constante de compilação (M)
- Raiz no header · 1 nó por I/O · nunca carrega tudo
- Free list no próprio arquivo (reaproveitamento)



Árvore B ($m=3$, 40 chaves) exportada pelo próprio código — o número é o RRN no disco

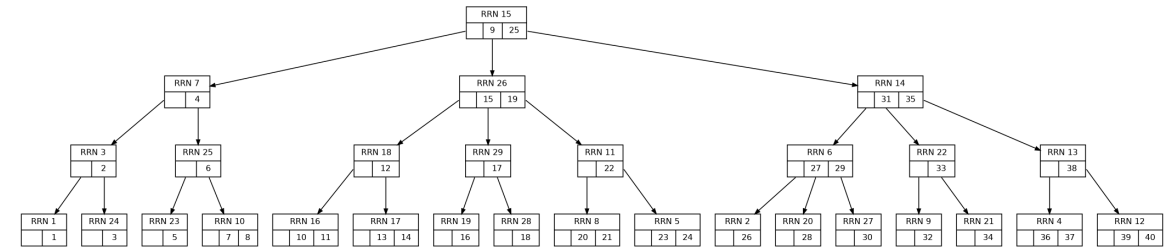
- Toda E/S passa por uma única classe
- readNode / writeNode = 1 registro de PAGE_SIZE
- Aqui mora o contador de acessos (métrica central)
- A árvore nunca é carregada inteira

- Remoção libera nó → vira nó LIVRE
- freeNode encadeia na lista (free_head no header)
- allocNode reusa antes de crescer o arquivo
- Ligável (setReuse) — medido no churn

- Busca m-way (mSearch)
- Inserção bottom-up com split (insertB)
- Remoção: sucessor, redistribuição e fusão (deleteB)
- Monitoramento: printTree · height · exportDot

Busca m-way

- Chaves dividem o universo em faixas
- Desce pelo ponteiro da faixa certa
- Custo $\approx$ altura da árvore
- Buscar 70: apenas 2 acessos



A busca segue um caminho da raiz à folha

- Bottom-up: acha a folha, insere em ordem
- Estourou ($> m-1$ chaves)? $\rightarrow$ split
- A mediana sobe ao pai; pode propagar e criar raiz
- É o que mantém a árvore balanceada

- Nó interno: troca pelo sucessor in-order
- Folha pode ficar em underflow
- Redistribuição: empresta de um irmão
- Fusão: funde com irmão + chave do pai

Resumo — o que o código faz

Núcleo — Árvore B em disco

- Ordem m parametrizável (M em compilação)
- 100% em disco: 1 nó por I/O
- Busca · inserção (split) · remoção (merge)
- Persistência por RRN + header

Extras

- Contador de acessos ao disco
- Free list ligável (setReuse)
- printTree · height · exportDot (Graphviz)
- Benchmark CSV + automação (run_all, make_tables)

Demonstração — o programa rodando

- Compilação → menu → CRUD + métricas
- $m=3$ e $m=5$: a árvore fica mais rasa
- Captura real da execução

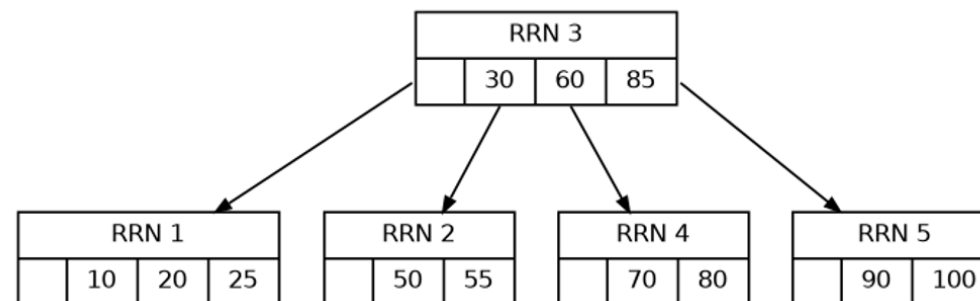
```
Arvore B (ordem m=5) - demo

$ ./btree demo5.bin

=== B-Tree (ordem 5) ===
... inserir / buscar / remover ...

> Imprimir arvore:
[RRN 1] (n=3) keys: 10 20 25
[RRN 2] (n=2) keys: 50 55
[RRN 4] (n=2) keys: 70 80
[RRN 5] (n=2) keys: 90 100
```

Grafo gerado ($m=5$)



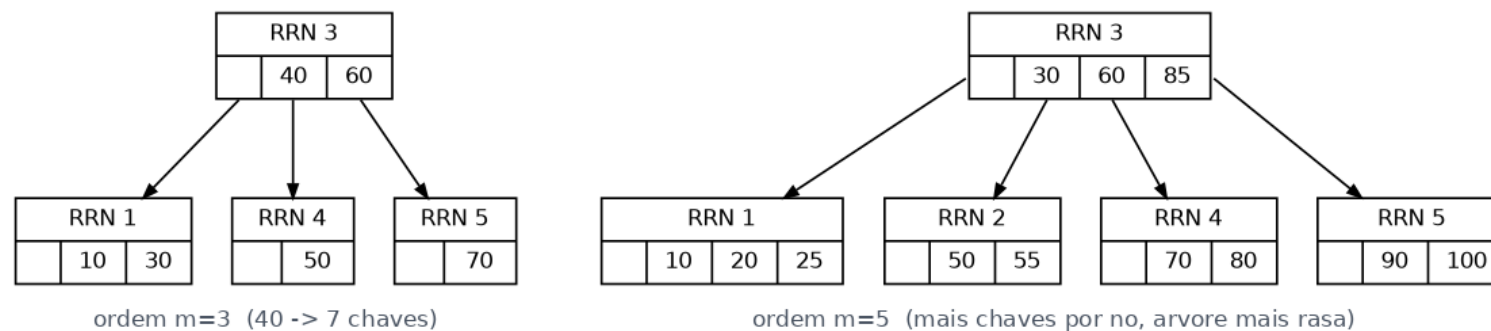
cada caixa = um nó em disco (RRN)
exportado pelo próprio programa -> exportDot() + Graphviz (dot)

make M=3/M=5 → menu interativo → grafo exportado pelo programa

Grafos gerados pelo programa

- Exportado pelo programa (Graphviz)
- Cada caixa = nó no disco (RRN)
- $m=5$: mais chaves por nó $\rightarrow$ árvore mais rasa

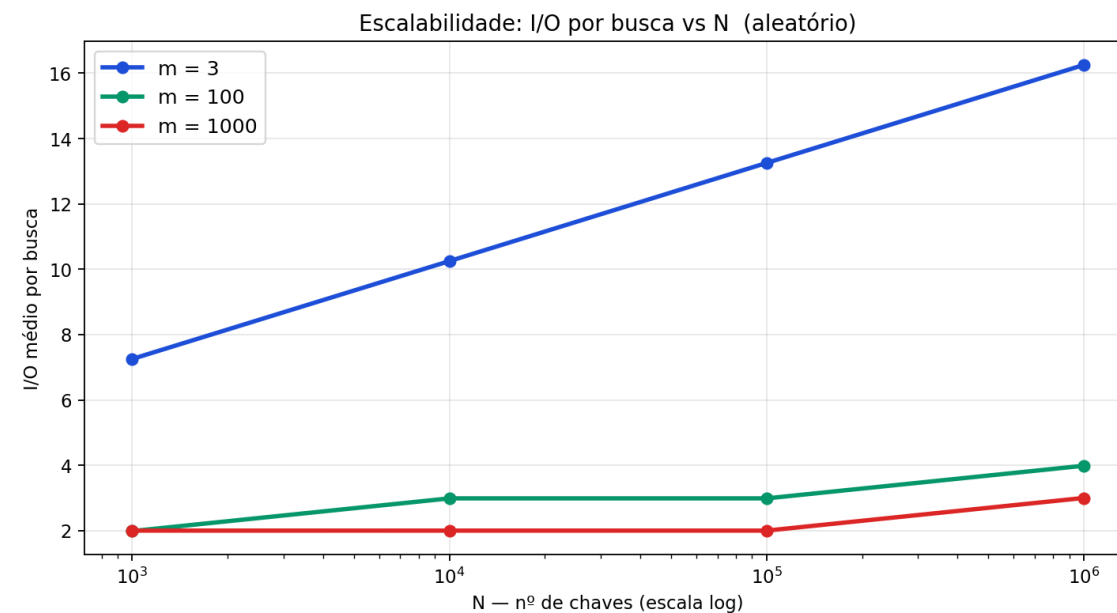
Grafos gerados pelo programa (exportDot + Graphviz)



Mesma sequência, duas ordens m — à direita a árvore é mais rasa

Os experimentos — a matriz de testes

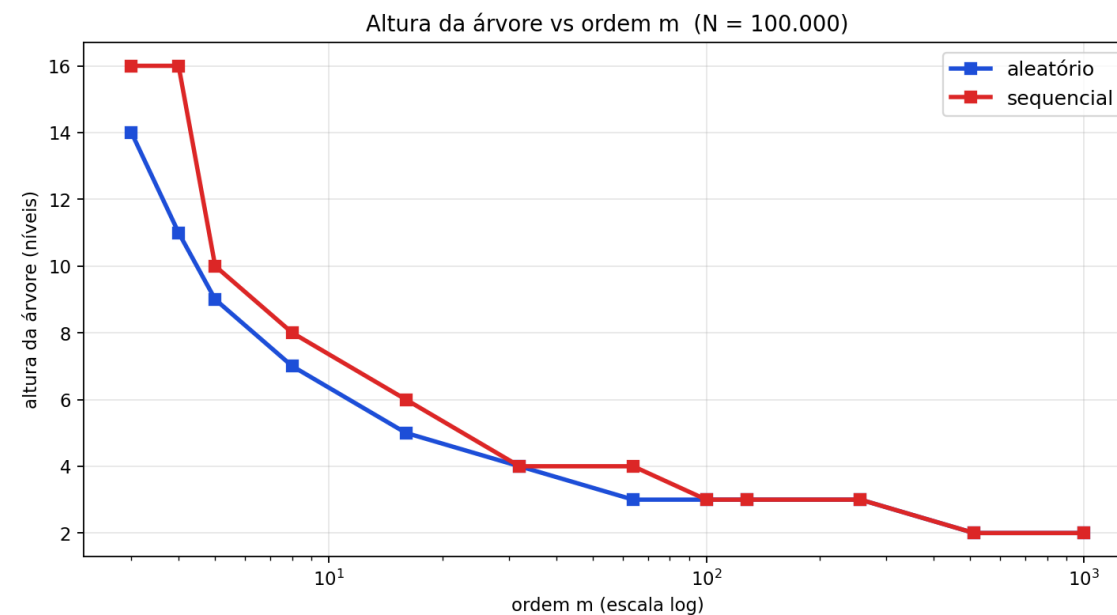
- Exp.1 — impacto da ordem m ($N=10^4$)
- Exp.2 — escala N (10^3 – 10^6)
- Exp.3 — ocupação com/sem reuso
- Exp.4 — CPU vs I/O
- Validação em 2 máquinas



- Driver não interativo → CSV de métricas
- Cada configuração reconstruída do zero
- Modos aleatório e sequencial
- getrusage: CPU usuário / sistema / espera de I/O
- 2 máquinas: Notebook × Titan (USP)

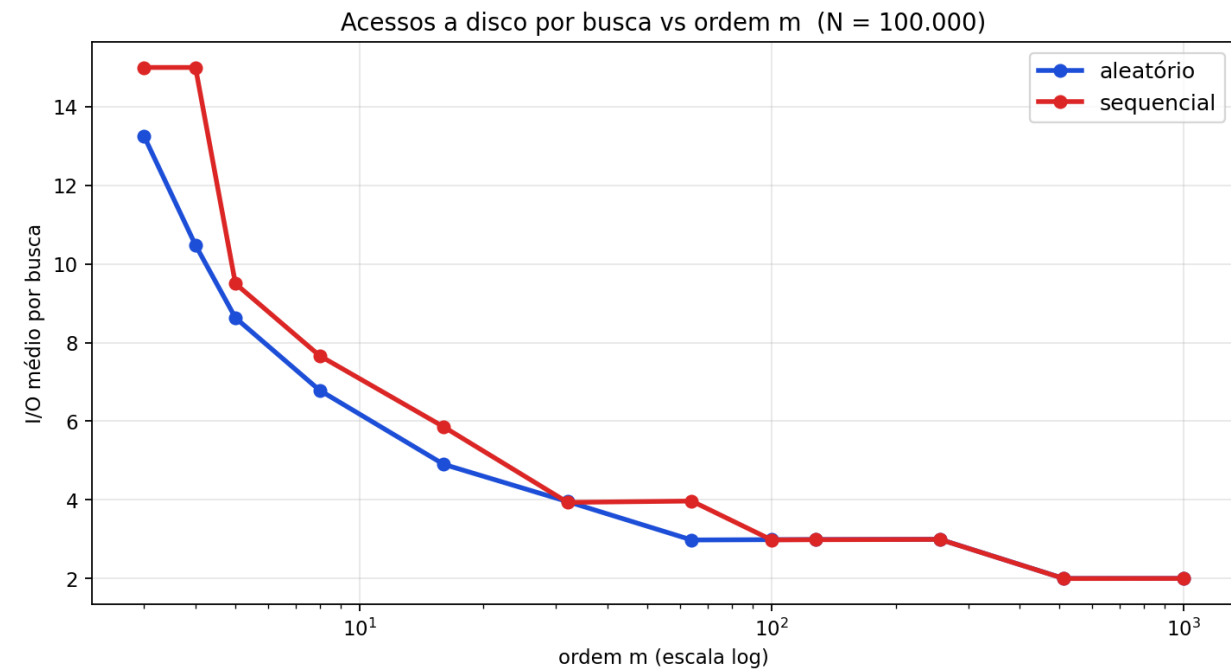
Métricas utilizadas

- Acessos ao disco — métrica central
- Altura ($\sim \log_m N$)
- Ocupação: nós, livres e bytes
- Tempo: wall · CPU (user/sys) · espera de I/O



Impacto da ordem m — I/O por busca (N=100000)

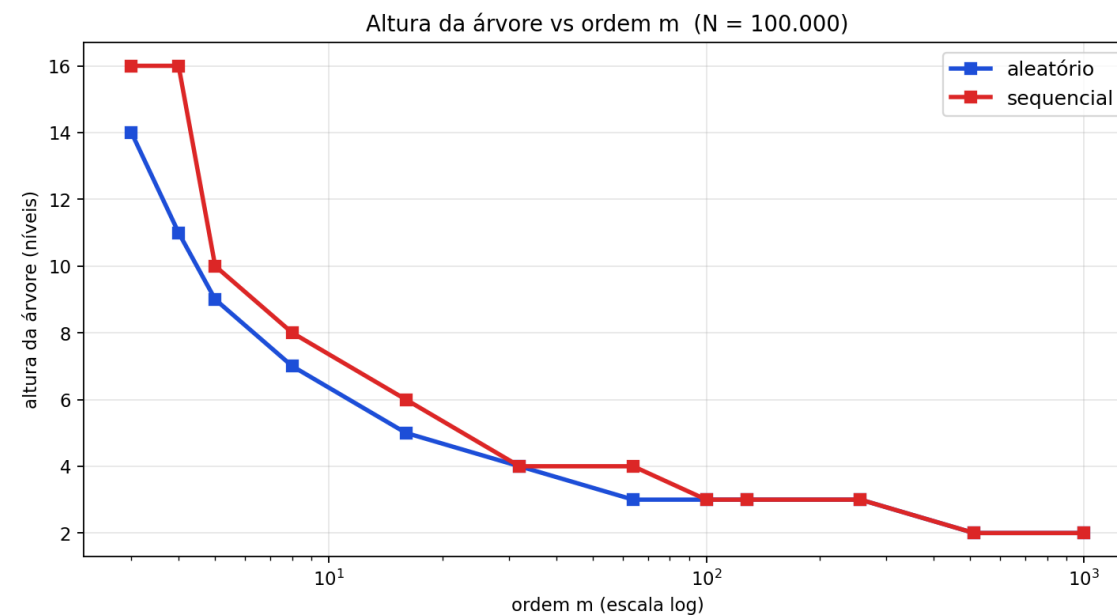
m	altura	busca I/O/op	inser. I/O/op	rem. I/O/op	arquivo (KB)
3	14	13.25	16.90	19.67	2,040
4	11	10.48	13.51	16.76	1,829
5	9	8.63	11.50	13.52	1,589
8	7	6.79	9.05	10.91	1,411
16	5	4.90	7.00	8.16	1,251
32	4	3.95	5.95	6.77	1,189
64	3	2.98	5.04	5.64	1,146
100	3	2.99	4.97	5.51	1,131
128	3	2.99	4.92	5.41	1,121
256	3	2.99	4.57	5.00	1,088
512	2	2.00	4.00	4.41	1,053
1000	2	2.00	3.99	4.34	1,016



I/O por busca cai de 13,25 ($m=3$, altura 14) para 2,00 ($m \geq 512$, altura 2). Ganho satura por volta de $m \approx 64-128$.

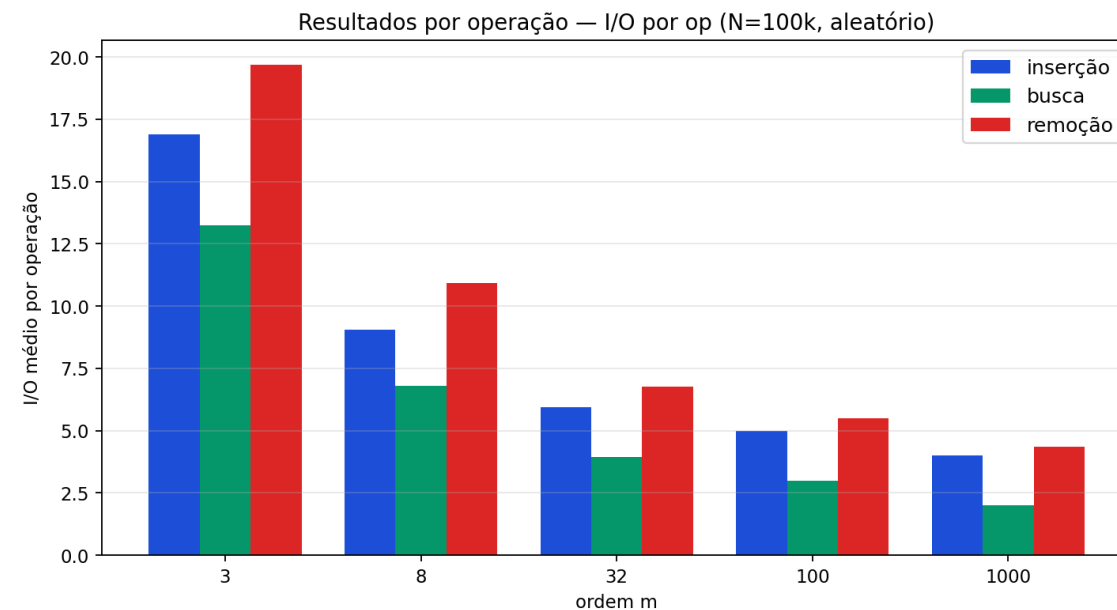
Impacto da ordem m — altura

- Altura $\approx$ acessos por busca
- $m=3 \rightarrow 14$ níveis (fundo demais)
- m cresce $\rightarrow$ altura despenca em degraus
- m grande $\rightarrow$ altura 2



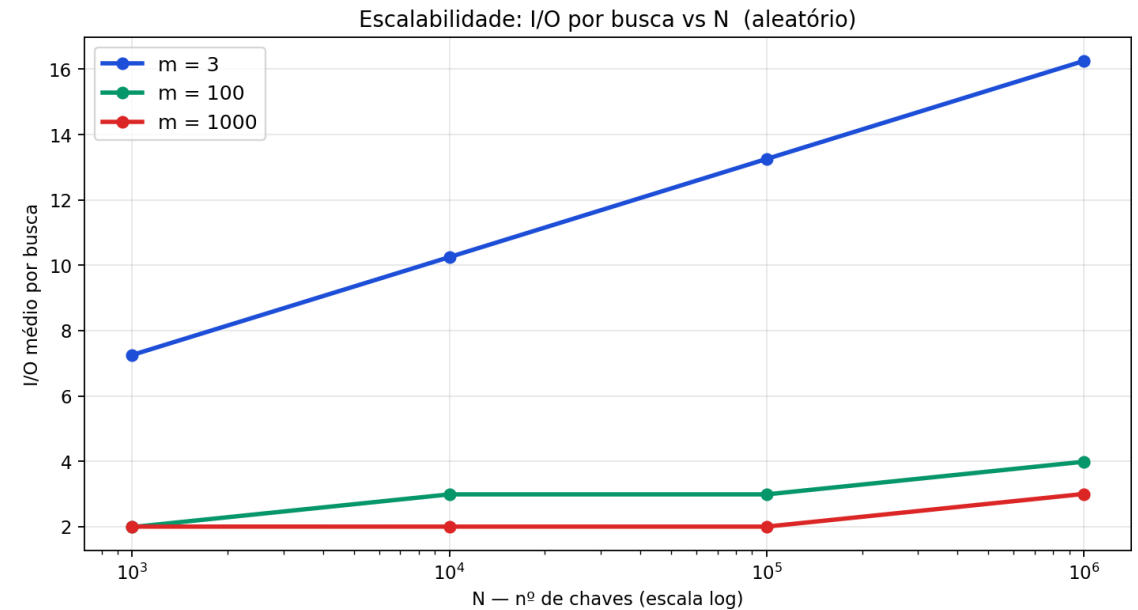
Resultados por operação

- Busca: a mais barata (raiz → folha)
- Inserção: split reescreve nós
- Remoção: a mais cara (redistribui / funde)
- Todas caem com m e saturam juntas



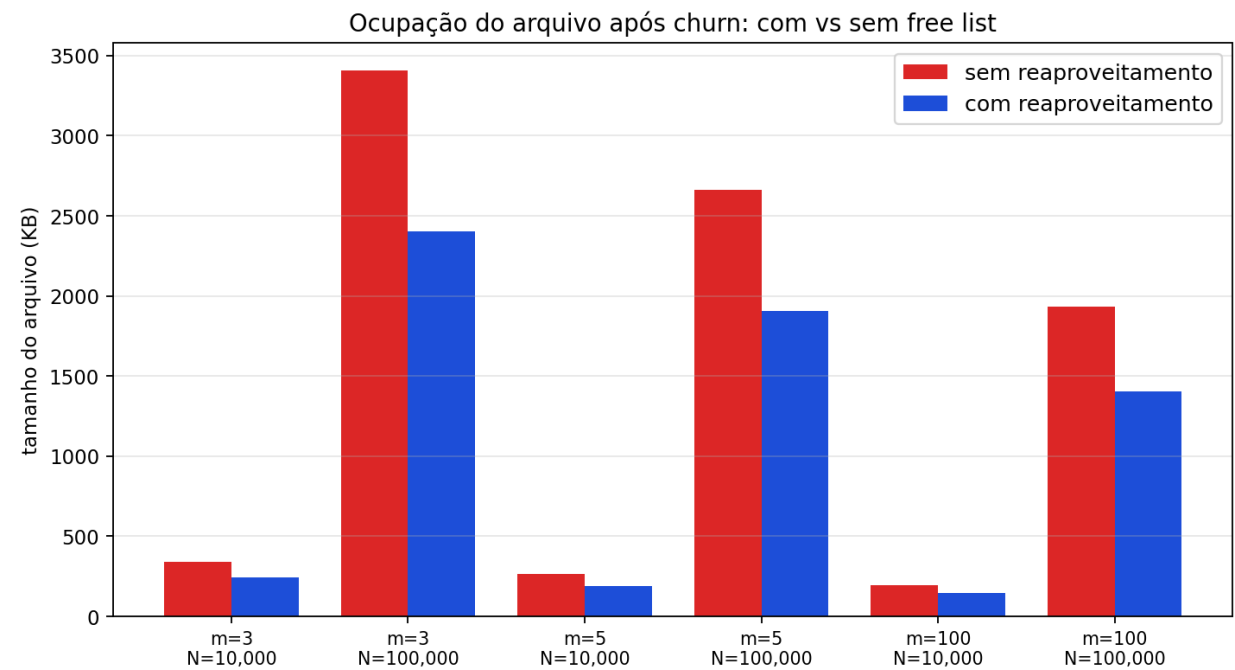
Escala do conjunto (mil → 1 milhão)

- N de mil a um milhão de chaves
- Acessos por busca quase não mudam
- Custo logarítmico na base m
- Escalável: cresce o dado, não o custo



Reaproveitamento de nós (churn)

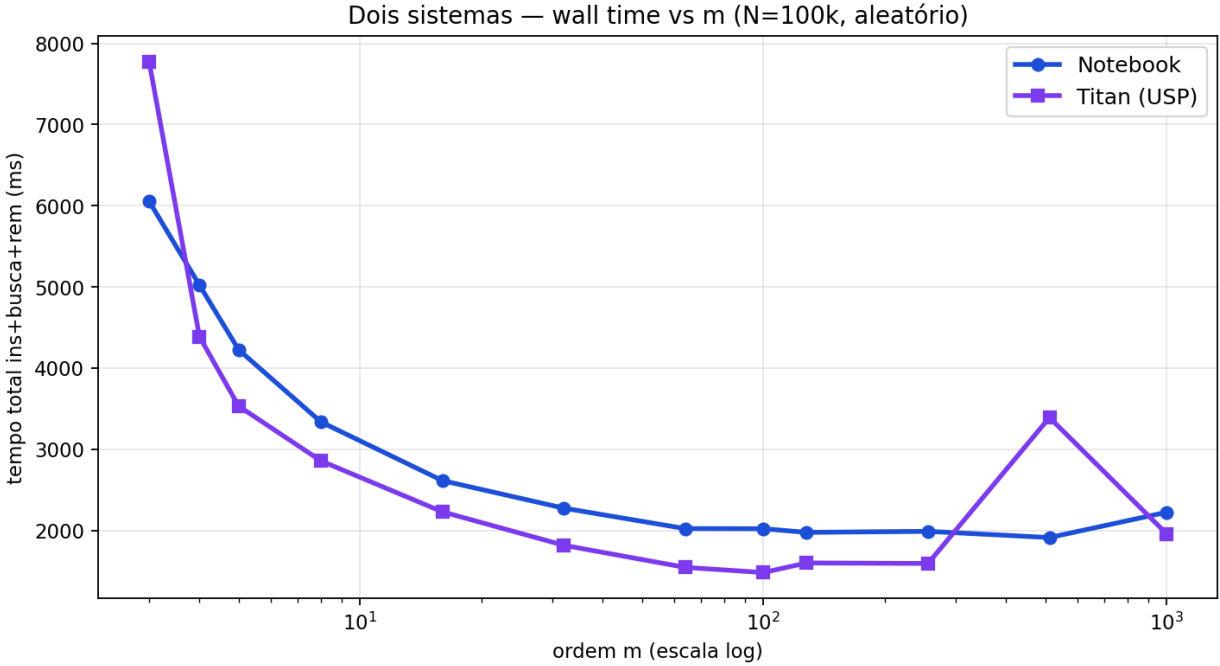
m	N	sem reuso (KB)	com reuso (KB)	economia
3	10,000	342	241	30%
3	100,000	3,407	2,404	29%
5	10,000	266	190	29%
5	100,000	2,663	1,905	28%
100	10,000	196	144	27%
100	100,000	1,931	1,404	27%



O reaproveitamento economiza ~27-30% do tamanho do arquivo neste workload.

Dois sistemas — tempo (wall)

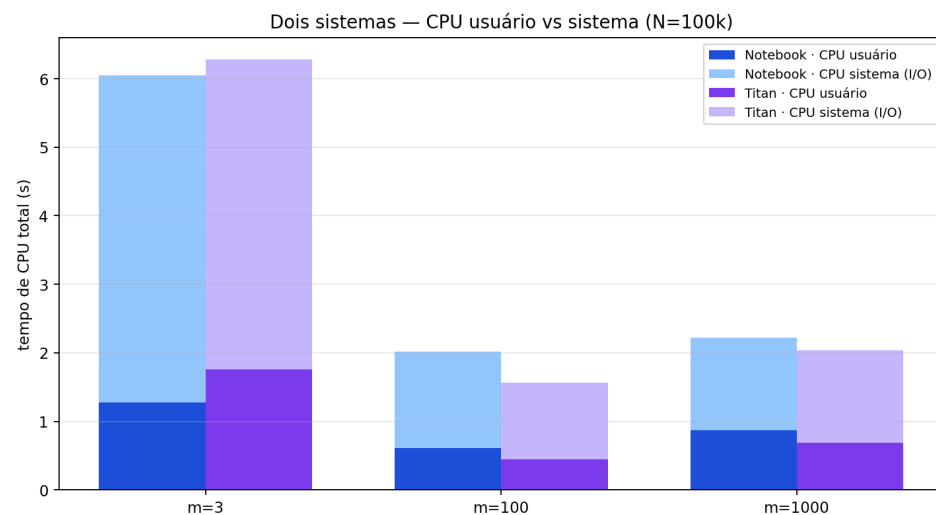
m	acessos disco	Notebook wall	Titan wall	speedup
3	4.982.252	6.05 s	7.78 s	0.78×
16	2.006.878	2.61 s	2.23 s	1.17×
100	1.346.869	2.02 s	1.48 s	1.36×
1000	1.033.255	2.22 s	1.96 s	1.14×



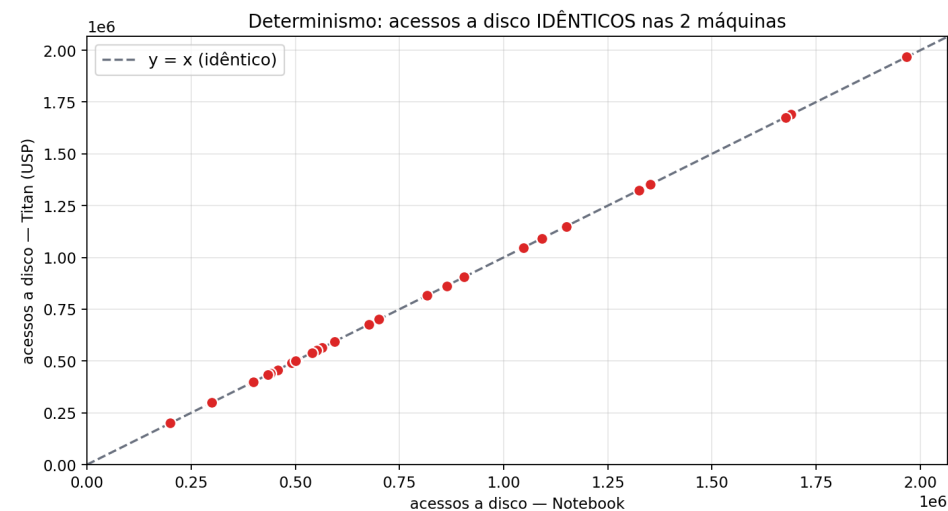
Mesmo programa, 2 máquinas: acessos a disco idênticos; só o tempo muda — a curva tem a mesma forma.

Dois sistemas — CPU e determinismo

- Tempo é quase todo CPU de sistema (I/O), não espera de disco
- Acessos idênticos: todos os pontos sobre $y=x$
- Determinística e portátil



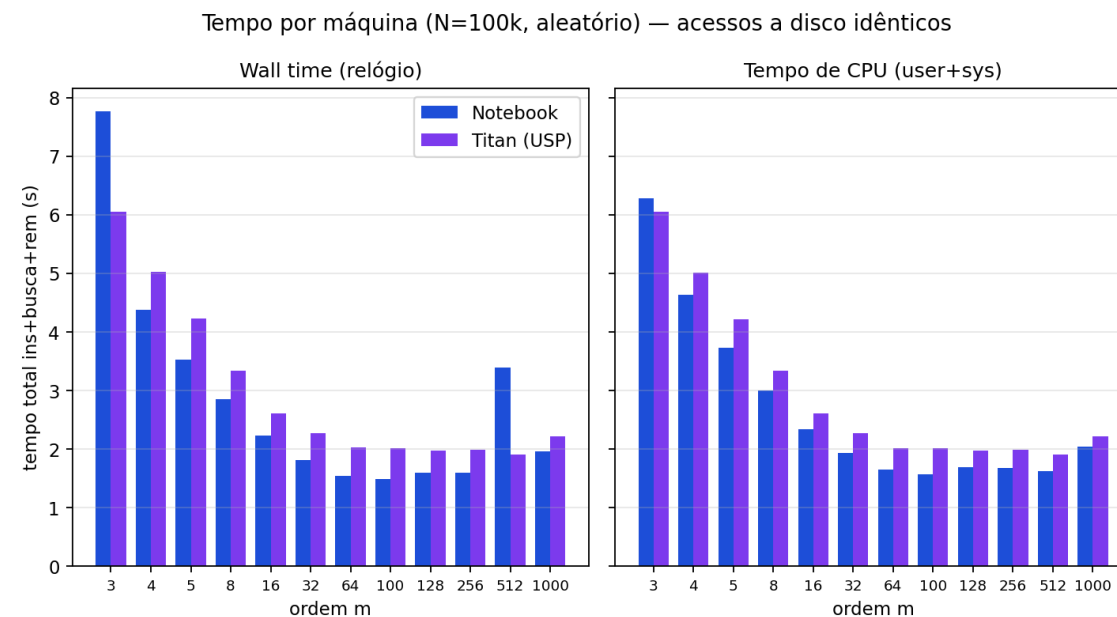
CPU usuário vs sistema (I/O) nas 2 máquinas



Acessos a disco idênticos — pontos sobre $y=x$

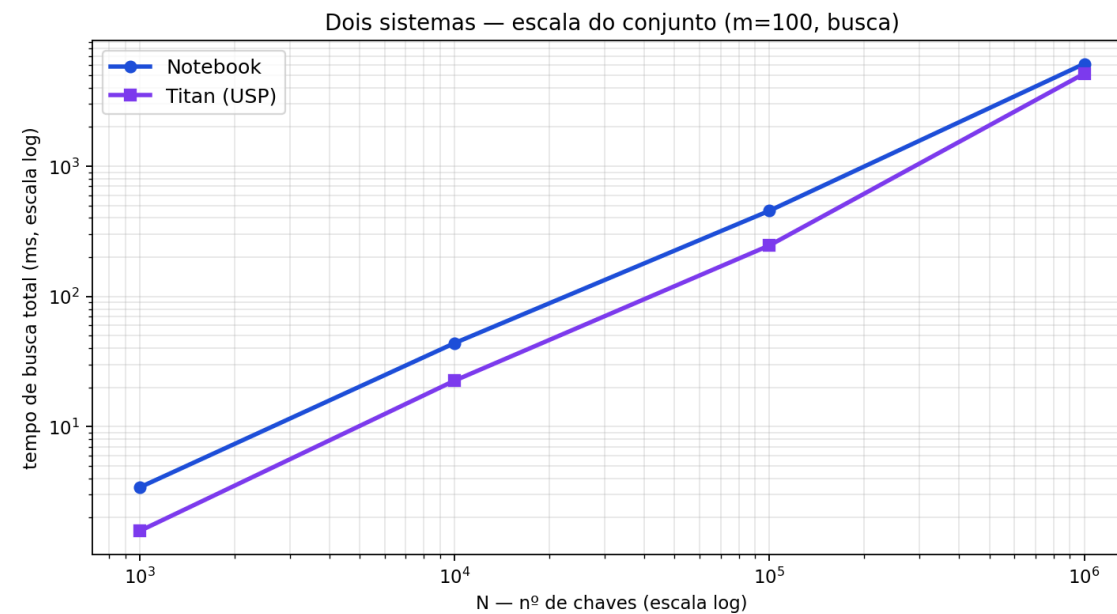
Dois sistemas — validação cruzada

- Notebook × Titan (USP)
- Acessos a disco idênticos
- Só o tempo varia
- Comparação via compare.py



Dois sistemas — escala do conjunto

- N até 1 milhão ($m=100$), nas 2 máquinas
- Tempo cresce devagar (logarítmico)
- Mesma tendência; Titan mais rápido
- Prova prática de escalabilidade



E se usássemos union no header? (sugestão do prof.)

Como está hoje

- Header e BNode são tipos separados
- E/S serializa campo a campo (memcpy)
- Header usa 1 página inteira (RRN uniforme)
- Nó livre esconde o próximo em $K[1]$

Com union

- Página única: header e nós no mesmo tamanho fixo
- Lê/escreve a página inteira → menos código
- FreeNode.next explícito (autodocumentado)
- Custo: portabilidade do binário (padding/endianness)

- Indexação 1-based (RRN 0 = header)
- Remoção: manter o path para reparar underflow
- eofbit do fstream → resolvido com clear()
- Serialização campo a campo é propensa a erro

Vantagens × Desvantagens

Vantagens

- Altura baixa → poucos acessos
- Sempre balanceada
- Ideal para disco e índices de SGBD
- Determinística + reuso de espaço

Desvantagens / custos

- Nós subocupados (~50% no sequencial)
- Remoção complexa
- m muito grande → mais CPU por nó
- Existe um m ótimo (tamanho de bloco)

- SGBDs: MySQL/InnoDB, PostgreSQL, Oracle
- Sistemas de arquivos: NTFS, APFS, ext4, Btrfs
- Chave-valor: SQLite, LMDB, BerkeleyDB
- Uso natural: índice de um banco em disco

Ferramentas utilizadas

Implementação & experimentos

- C++17 (g++ -O2) · GNU Make
- std::fstream (binário, registros fixos)
- Graphviz (grafos) · Python + matplotlib
- python-pptx / reportlab (slides)

Claude (IA) — apoio, não autor

- Assistente de revisão e ferramentas acessórias
- A lógica da Árvore B é autoral
- Apoiou menu, exportação de grafos e scripts
- Entendimento e decisões são nossos

- Árvore B 100% em disco, 1 nó por I/O
- I/O por operação cai com m até saturar ($\sim 64-128$)
- Free list economiza $\sim 27-30\%$ de espaço
- Determinística: acessos idênticos em 2 máquinas

- Comer, D. (1979). The Ubiquitous B-Tree. ACM Computing Surveys.
- Knuth, D. The Art of Computer Programming, Vol. 3.
- Bayer, R. & McCreight, E. (1972). Large Ordered Indexes.
- Baranauskas, J. A. Slides AED-PG-2026 (Árvores B). DCM / USP.
- Obrigado! Perguntas?